



AI-Powered DevOps (AIOps)

Automating CI/CD Pipelines, Code Review, and Cost Optimization

A Professional Technical Report

Course: Technical and Business Writing

Assignment 2: Professional Technical Report Formatting

Department: Department of Computer Science (UBIT), University of Karachi

Name	Seat No.
Humaima Moin	B23110006043
Mehak Tariq	B23110006066
Neha Rehan	B23110006131
Sana Ghulam Rasool	B23110006147
Sania Khan	B23110006148
Serena Salman	B23110006151
Muhammad Huzaifa	B23110006102
Syed Farhan Iqbal	B23110006160
Hamza Rizwan	B23110006038

Submitted to: Miss Farheen Faisal

Date: 20 September 2026

Table of Contents

- 1 Introduction
 - 1.1 What Is DevOps?
 - 1.2 Why Organizations Use DevOps
 - 1.3 Problems with Traditional DevOps
 - 1.4 Role of Automation
- 2 What Is AIOps?
 - 2.1 Definition of AIOps
 - 2.2 AI + DevOps
 - 2.3 Machine Learning in Software Operations
 - 2.4 How AIOps Analyzes Development and Operations Data
- 3 Why AIOps Is Important
- 4 Traditional DevOps vs. AIOps
- 5 AI-Powered CI/CD Pipeline Automation
 - 5.1 From Automated to Intelligent Pipelines
 - 5.2 Key AI Techniques in CI/CD
 - 5.3 Benefits and Challenges
- 6 Intelligent Code Review in AI-Powered DevOps
 - 6.1 How Intelligent Code Review Works
 - 6.2 Technical Analysis Methods
 - 6.3 Example Application
 - 6.4 Advantages and Limitations
- 7 AI-Based Cost Optimization
 - 7.1 Cloud Cost and Pipelines
 - 7.2 Cost Optimization in the Pipeline
 - 7.3 Measuring the Results
 - 7.4 Advantages and Limitations
- 8 Implementation, Case Studies, and Future Trends
 - 8.1 Implementation Architecture
 - 8.2 Empirical Case Study Analysis
 - 8.3 Future Implementations and Emerging Directions
- 9 Conclusion
- References

1 Introduction

Modern software is released more often, runs on more complex infrastructure, and produces far more data than it did a decade ago. Applications are split into microservices, deployed to cloud platforms and containers, and updated many times a day. Keeping such systems fast and reliable is more than a team can do by hand. This report explains DevOps and the problems it leaves unsolved, then introduces AIOps, which applies artificial intelligence to software development and operations, and compares the two approaches (Sections 2 to 4). It then examines AI-powered CI/CD pipeline automation (Section 5), intelligent code review (Section 6), and cost optimization (Section 7), followed by an implementation architecture, case studies, and future directions (Section 8) and a conclusion (Section 9).

1.1 What Is DevOps?

DevOps is a set of cultural practices, processes, and tools that bring software **development (Dev)** and IT **operations (Ops)** together (Amazon Web Services, n.d.-b; Kim et al., 2016). In the older model, developers wrote code and handed it to a separate operations team to deploy and run. In DevOps, both groups share responsibility for the whole lifecycle: planning, coding, building, testing, releasing, deploying, operating, and monitoring. It works as a continuous loop, because what is learned from monitoring in production feeds back into the next round of planning. The core ideas are close collaboration between teams, continuous integration and continuous delivery (CI/CD), automation of repetitive work, and constant monitoring and feedback.

1.2 Why Organizations Use DevOps

Organizations adopt DevOps because it improves both the speed and the quality of software delivery (Amazon Web Services, n.d.-b; Kim et al., 2016). The main benefits are:

- **Faster delivery:** small, frequent releases get new features and fixes to users sooner.
- **Better collaboration:** shared goals reduce hand-off delays between development and operations.
- **Higher quality:** automated testing and early feedback catch defects before they reach production.
- **Greater stability:** repeatable deployments lower the chance of failed releases and speed up recovery.
- **Consistency at scale:** standard pipelines and infrastructure defined as code let teams manage many services in the same way.

1.3 Problems with Traditional DevOps

DevOps improved on the siloed approach, but as systems have grown in scale and complexity (Notaro et al., 2021), several limits of the traditional model have become clear:

- **Data overload:** hundreds of services produce logs, metrics, traces, and alerts faster than engineers can read them.

- **Alert fatigue:** static thresholds generate false or duplicate alerts that bury real problems.
- **Reactive troubleshooting:** problems are often noticed only after users are affected, and root-cause analysis is slow and manual.
- **Rigid rule-based automation:** scripts and rules do only what they were written to do and fail when conditions change.
- **Human-heavy processes:** code review, incident triage, and capacity planning depend on people, which creates bottlenecks and inconsistent results.
- **Static resource allocation:** capacity fixed in advance leads either to wasted cost (over-provisioning) or to outages (under-provisioning).

1.4 Role of Automation

Automation is the backbone of DevOps. Tools take over repetitive tasks such as building code, running tests, provisioning servers, and deploying releases (Amazon Web Services, n.d.-b), usually chained into a CI/CD pipeline like the one in Figure 2. Traditional automation, however, is rule-based: it carries out instructions exactly as written, but it does not learn from past events, spot patterns in data, or decide what to do in a new situation. This gap is what AIOps is designed to fill.

2 What Is AIOps?

2.1 Definition of AIOps

AIOps (Artificial Intelligence for IT Operations) is the use of artificial intelligence, machine learning, and big-data analytics to automate and improve the way software systems are developed, deployed, and operated. An AIOps platform collects data from many sources, finds patterns and anomalies in it, and then recommends or carries out actions with little or no human intervention (Amazon Web Services, n.d.-a; IBM, n.d.). The field covers a wide range of tasks, from resource management and scheduling to failure prediction, anomaly detection, and remediation (Bogatinovski et al., 2021; Notaro et al., 2021).

2.2 AI + DevOps

AIOps does not replace DevOps; it extends it. DevOps supplies the culture, the pipelines, and a large stream of data, and AI adds the ability to analyze that data, learn from it, and act on it at every stage of the lifecycle: estimating effort and risk when planning, suggesting code and supporting AI-assisted code review, prioritizing the tests most likely to fail, scoring the risk of a release and triggering an automatic rollback, and detecting anomalies, predicting incidents, and resolving common issues during operation.

2.3 Machine Learning in Software Operations

Several machine learning techniques are used in AIOps, each suited to a different kind of problem:

- **Supervised learning:** models trained on labelled historical incidents classify the severity of a new issue or predict a likely failure.
- **Unsupervised learning:** clustering and anomaly detection learn what normal behaviour looks like and flag deviations without labelled examples.
- **Time-series forecasting:** predicts future load, memory or disk usage, and capacity needs from past trends.
- **Natural language processing:** reads logs, tickets, and documentation, groups similar incidents, and suggests fixes based on earlier resolutions.

2.4 How AIOps Analyzes Development and Operations Data

An AIOps platform works through a repeating cycle (IBM, n.d.), illustrated in Figure 1:

1. **Collect:** gather logs, metrics, traces, code commits, deployment events, and incident tickets from across the toolchain.
2. **Process:** clean, normalize, and aggregate the data so that different sources can be compared.
3. **Analyze:** learn a baseline of normal behaviour, detect anomalies, and correlate related events so that hundreds of alerts collapse into a single incident with a likely root cause.
4. **Predict:** forecast failures or capacity shortages before they affect users.
5. **Act:** send prioritized alerts, recommend fixes, or apply them automatically, for example by restarting a service or scaling resources.
6. **Learn:** use the outcome of each action as feedback to improve the models over time.

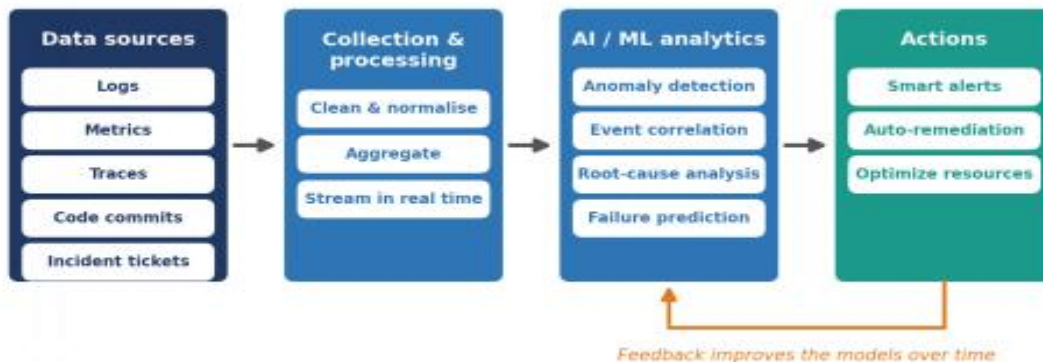


Figure 1: The AIOps Workflow From Data Collection to Automated Action

3 Why AIOps Is Important

AIOps offers five main benefits:

- **Faster development:** AI-assisted code review, smarter test selection, and automatic analysis of failed builds shorten feedback loops, so developers spend less time waiting for results or hunting through logs.

- **Fewer human errors:** AIOps applies the same analysis every time, and it can flag risky changes or misconfigurations before they reach production.
- **Automated decision-making:** AIOps can decide, or recommend, which alerts deserve attention, whether to roll back a faulty release, and when to add capacity, while humans keep oversight of high-impact actions.
- **Improved reliability:** detecting early warning signs lets teams fix many problems before users notice them (Notaro et al., 2021), which lowers the mean time to detect (MTTD) and the mean time to repair (MTTR).
- **Better resource utilization:** forecasting demand and adjusting capacity automatically avoids both idle, expensive infrastructure and overloaded systems.

4 Traditional DevOps vs. AIOps

Table 1 summarizes the main differences between traditional DevOps and AIOps.

Table 1

Comparison of Traditional DevOps and AIOps

Traditional DevOps	AIOps
Rule-based automation	AI/ML-based automation
Manual analysis	Automated analysis
Manual identification of some problems	Predictive problem detection
Static resource allocation	Dynamic optimization
Human-heavy code review	AI-assisted code review

The key shift is from following fixed rules and reacting to problems to learning from data and anticipating them. AIOps builds on DevOps rather than replacing it: the pipelines and practices stay, and AI adds analysis, prediction, and decision support, with humans remaining in the loop for high-impact decisions.

5 AI-Powered CI/CD Pipeline Automation

Continuous Integration and Continuous Delivery (CI/CD) is the practice of automatically building, testing, and releasing software every time developers change the code (Humble & Farley, 2010). Shahin et al. (2017) reviewed the approaches and tools used for CI/CD and found that, alongside its benefits, teams struggle with long build times, complex test suites, and difficult failure diagnosis. As a project grows, a pipeline that only follows fixed, hand-written rules becomes slow, costly, and hard to maintain.

AI-powered CI/CD addresses this by applying machine learning to the data that pipelines already produce, such as commits, test results, build logs, and deployment metrics. Instead of only executing scripts, the pipeline can predict, prioritize, and decide. This is one of the main application areas of the AIOps approach introduced in Section 2 (Dang et al., 2019; Notaro et al., 2021).

5.1 From Automated to Intelligent Pipelines

A traditional pipeline runs the same stages in the same order for every change: build, test, package, and deploy. Forsgren et al. (2018) show that high-performing teams deploy more often and recover from failures faster, so speed and stability must improve together. AI supports this by adding a decision layer on top of each stage. Figure 2 shows the main stages, the AI component attached to each one, and the feedback loop that keeps the models up to date.

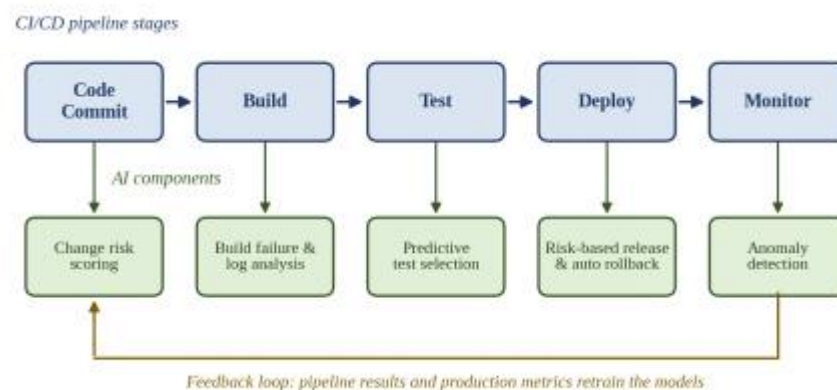


Figure 2: AI Components Across the CI/CD Pipeline

5.2 Key AI Techniques in CI/CD

Predictive test selection. Large projects can have thousands of tests, and running all of them for every small change wastes time and computing resources. Machalica et al. (2019) describe a system at Facebook that learns from historical test outcomes to select only the tests most likely to fail for a given change. In production, this approach cut the infrastructure cost of testing by half while still reporting over 99.9% of faulty changes to developers.

Automated failure analysis. When a build fails, developers usually read long logs to find the cause. Machine learning models can classify failures, group similar error messages, and separate flaky tests (tests that pass or fail without any code change) from real defects. Notaro et al. (2021) survey AIOps methods for failure management, including failure prediction and root-cause analysis, which apply directly to build and test failures in a pipeline.

Intelligent deployment and rollback. After a release, monitoring data such as error rate and response time can be analyzed by anomaly detection models. If the metrics move away from their normal baseline, the pipeline can pause a gradual rollout or roll back automatically, before most users are affected. This shortens the time between a bad release and its correction (Dang et al., 2019).

5.3 Benefits and Challenges

A traditional pipeline runs all tests or a fixed rule-based set, relies on manual reading of logs, uses fixed approval gates and manual rollback, and is improved by updating rules by hand. An AI-powered pipeline instead selects the tests most likely to fail from historical data, classifies failures and suggests probable causes, makes risk-based deployment decisions with automatic rollback on anomalies, and retrain its models on new pipeline data. The main benefits are therefore faster feedback for developers, lower testing cost, less manual log analysis, and fewer failed releases. The challenges are that models need large amounts of clean historical data, a wrong prediction may skip a test that would have caught a bug (so teams usually keep a safety net such as periodic full test runs), decisions can be hard to explain, and the models must be monitored and retrained. Human oversight therefore remains necessary. Section 6 builds on this discussion by examining intelligent code review, which applies similar techniques at the level of the code itself.

6 Intelligent Code Review in AI-Powered DevOps

Intelligent code review is an AI-assisted software engineering process that examines source-code changes before they are merged into a shared repository, placing automated reasoning directly inside the delivery pipeline. Traditional review depends mainly on human reviewers to identify defects, security problems, maintainability issues, and violations of project conventions. AI-assisted review adds automated analysis that can examine a pull request, explain possible problems, and suggest corrective actions. Platforms such as GitHub Copilot support AI-based pull-request review while emphasizing that developers should validate the generated feedback through human review and other checks (GitHub, 2026a).

6.1 How Intelligent Code Review Works

An intelligent code-review system normally operates as a multi-stage pipeline. When a developer commits a change or opens a pull request, the system collects the code diff and relevant repository context, and it may combine deterministic tools such as compilers, linters, tests, and security scanners with an AI model. The AI component analyzes the change in context and generates findings that are prioritized according to confidence and potential impact. Figure 3 shows that AI review is not simply a chatbot examining isolated code: context collection and deterministic validation provide evidence that helps the AI produce more useful findings.



Figure 3: *Simplified Workflow of an AI-Assisted Code-Review Pipeline*

6.2 Technical Analysis Methods

Defect and logic detection. AI models can inspect changed code for logical errors that syntax checking does not detect. For example, a function may compile correctly but fail when it receives an empty list, an unexpected value, or an unauthorized request. An intelligent reviewer can examine conditions, control flow, error handling, and edge cases and explain a possible failure scenario; the finding is more reliable when it is supported by a generated test or an existing test failure.

Security and maintainability. Security review can identify suspicious patterns such as missing authorization checks, unsafe input handling, exposed credentials, and insecure API usage, while maintainability review can flag unnecessary duplication, excessive complexity, inconsistent naming, and deviations from repository conventions. However, security findings should be validated with established security scanners and human expertise. NIST guidance emphasizes that AI systems should be evaluated for reliability, security, transparency, privacy, and other risk characteristics rather than being treated as automatically trustworthy (Tabassi, 2023; Autio et al., 2024).

Finding prioritization. A practical system should not present every observation with the same priority: high-confidence defects that can cause data loss, security compromise, or application failure deserve more attention than minor style suggestions. GitHub's AI code-review documentation similarly describes severity-based findings and different levels of review effort (GitHub, 2026c). Table 2 gives an example of how findings can be prioritized.

Table 2

Example Prioritization of Intelligent Code-Review Findings

Finding	Evidence	Priority
Security / authorization flaw	Scanner + code context	High
Logic defect	AI reasoning + test	High
Maintainability issue	Repository context	Medium
Style / naming issue	Rule or convention	Low

6.3 Example Application

Consider an e-commerce API with an order-cancellation endpoint that a developer modifies so that a logged-in user can cancel an order by supplying its identifier. The code may compile and pass basic tests, but an AI reviewer examining the authentication middleware, order model, and existing tests could identify a missing ownership check: authentication only confirms the user's identity, whereas authorization must also verify that the selected order belongs to that user. It could then suggest a test in which User A attempts to cancel User B's order. The proposed test can be executed through the normal CI pipeline, giving the development team concrete evidence before the change is merged.

6.4 Advantages and Limitations

Intelligent code review provides faster feedback, consistent first-pass analysis, scalable review support, natural-language explanations, and reduced workload for human reviewers, especially for repetitive checks and early in the pull-request lifecycle. GitHub documents workflows in which AI can review draft changes and review substantial updates again after modifications (GitHub, 2026b).

The approach also has important limitations. AI systems can produce false positives, miss real defects, misunderstand business requirements, or generate an incorrect explanation or patch, and they may require access to sensitive source code and add cost and latency. AI review should therefore be used as a decision-support layer rather than the sole authority for merging code, and human reviewers remain responsible for architecture, business requirements, security exceptions, and high-impact engineering decisions. The central principle is controlled automation: AI provides evidence, explanations, and candidate fixes, while final responsibility remains with the engineering team.

7 AI-Based Cost Optimization

7.1 Cloud Cost and Pipelines

Armbrust et al. (2010) explain that the main attraction of the cloud is elasticity: a team does not need to buy hardware in advance, and it can avoid overprovisioning, which wastes money, and underprovisioning, which loses users. The other side is that a team pays for everything it uses, including waste, such as a slow test suite, a build server that is bigger than needed, or a test environment left on over the weekend. Cloud services are usually billed in US dollars, so this waste is felt even more by a small team in Pakistan. Cost optimization therefore means using only the resources that each change really needs.

7.2 Cost Optimization in the Pipeline

Pipeline waste usually comes from three places: running more tests than a change needs, running the pipeline again and again for changes that were already going to fail or were replaced by a newer commit, and leaving resources idle or oversized, for example a test environment that stays on overnight.

The cost optimizer reads pipeline usage and cloud billing data and suggests changes such as smaller runners or automatic shutdown of idle environments; its results go back into the system as feedback, so the models improve as more data arrives. For the first problem, predictive test selection uses the changed files and past failures of similar changes to run the most relevant tests first, while the full suite still runs on a schedule, for example every night. For the second, the code review agent acts as an early gate: if it finds a serious problem, the pipeline stops before spending build minutes. For the third, the optimizer shuts down temporary environments after a period of inactivity and chooses runner sizes from past usage, in line with the principle of Armbrust et al. (2010) to use only the

resources that are needed. Table 3 compares a traditional pipeline with one that has a cost optimizer at each stage, and Figure 4 shows how each source of waste is handled.

Table 3
Comparison of a Traditional Pipeline and a Pipeline With a Cost Optimizer

Stage	Traditional pipeline	With cost optimizer	Main benefit
Testing	Full test suite runs on every change	Tests are chosen using the change and past failures	Shorter runs
Failing changes	Full pipeline runs before a serious problem is found	Review agent stops the pipeline early	Fewer wasted runs
Build resources	Same runner size for every job	Runner size follows past usage	Less wasted compute
Test environments	Kept running until someone shuts them down	Shut down automatically when idle	Lower cloud bill

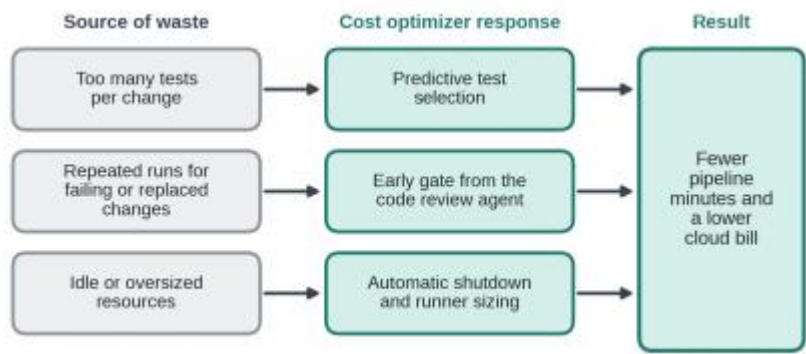


Figure 4: Sources of Pipeline Waste and How the Cost Optimizer Responds

7.3 Measuring the Results

A saving should be judged with numbers, not with feelings. We suggest tracking the four measures from Forsgren et al. (2018), which are how often the team deploys, how long a change takes to reach production, how quickly it recovers from a failure, and how often a change causes a failure, plus two of our own: pipeline minutes used per month and cloud cost per deployment. If deployment frequency rises and cost per deployment falls while the change failure rate stays the same, the optimizer is doing its job; if the failure rate rises, the team should check whether test selection is skipping tests it should not.

7.4 Advantages and Limitations

Advantages. Fewer wasted runs, shorter test time, and fewer idle environments lower the cloud bill; shorter pipeline runs mean developers wait less for results; and runner sizes that follow past usage mean the team pays only for what each job needs.

Limitations. The tool has its own cost, because every model call costs money and this can reduce the savings if the agent is used carelessly. Wrong test selection is a risk, because a real bug can pass through to production. Models also need fresh data as the codebase changes, and someone in the team must look after the setup.

8 Implementation, Case Studies, and Future Trends

8.1 Implementation Architecture

An AI-Powered CI/CD pipeline operates as a closed-loop system where machine learning models and Large Language Models (LLMs) intersect with source code management (SCM), automated testing, and cloud infrastructure orchestration. It consists of three key technical subsystems.

Intelligent code review and security analysis engine. Instead of relying on a single model, modern architectures use **multi-agent LLM orchestration**, with specialized agents working in parallel (e.g., dedicated Code Quality, Security/OWASP, and Logic Bug agents). **Retrieval-Augmented Generation (RAG)**, using vector databases such as FAISS, indexes enterprise coding standards, past Pull Request (PR) reviews, and repository histories, which grounds the LLM's feedback, minimizes hallucinations, and avoids redundant comments on recurring patterns. AI models also ingest the outputs of traditional SAST/DAST tools (e.g., SonarQube, Semgrep), filtering out false positives and framing actionable code modifications directly inside PR comments.

Predictive test selection and build optimization. Regression risk classifiers, which are supervised learning models (e.g., Random Forests, Gradient Boosted Trees), evaluate incoming code changes against past commit histories and build failures. Through **selective execution**, the engine schedules only high-risk test suites for a given change, reducing pipeline execution times without sacrificing code coverage.

FinOps-as-code and dynamic cloud cost optimization. Through **predictive workload placement**, Multi-Agent Systems (MAS) calculate the cost-to-performance ratio across cloud vendors in real time and place workloads on low-cost spot instances or lower-tier compute nodes based on deployment urgency. Through **resource right-sizing**, machine learning telemetry monitors CPU/RAM utilisation during runtime and updates Infrastructure-as-Code (IaC) definitions (such as Kubernetes HPA limits or Terraform templates) prior to deployment (Cogo et al., 2025).

8.2 Empirical Case Study Analysis

Recent peer-reviewed literature and empirical industry implementations validate the operational benefits of AI-infused CI/CD pipelines across reliability, cycle time, and cloud spend (Kakarla, 2024; Mamun, 2024).

Case Study 1: Multi-agent LLM frameworks for code review. **Focus:** mitigating human review latency and contextual irrelevance in pull requests. **Findings:** multi-agent collaboration, where specialized agents simultaneously evaluate maintainability, OWASP top-10 security guidelines, and logic errors, yielded a **30% reduction in end-to-end commit-to-release cycle times**. FAISS-based vector storage for developer feedback significantly reduced redundant or inaccurate feedback, increasing developer approval rates of AI-generated suggestions.

Case Study 2: Unified cloud cost optimization in multi-cloud CI/CD. **Focus:** empirical validation of AI-driven workload placement and policy-as-code fiscal governance within multi-cloud pipelines. **Findings:** adopting the Unified Theory of Cloud Cost Optimization (UTCCO) using Multi-Agent Systems demonstrated a **31% to 37.5% reduction in total cloud spending** through high-speed bidding on spot instances and automated right-sizing. Simultaneously, self-healing automated feedback loops reduced post-deployment system failure rates by **45%** (Kakarla, 2024).

Table 4 summarizes the reported impact of an AIOps-enabled pipeline on key performance indicators (KPIs).

Table 4

Key Performance Indicator Impact of AIOps-Enabled CI/CD Pipelines

Metric	Traditional CI/CD pipeline	AIOps-enabled CI/CD pipeline	Primary driver
PR feedback latency	Hours to days (human dependency)	< 5 minutes	Automated multi-agent LLM analysis
Build/test duration	Static execution (100% test suite)	35%–50% faster	Predictive ML test selection
False-positive security alerts	High (20%–40% noise)	Low (< 5% noise)	LLM contextual filtering over SAST output
Cloud infrastructure waste	High (static provisioning)	Reduced by 30%–38%	Dynamic right-sizing and spot orchestration

8.3 Future Implementations and Emerging Directions

As AIOps frameworks mature within software engineering, research points toward several next-generation developments:

- 1. Agentic Code Repair (Autonomous Self-Healing Pipelines).** Future CI/CD control planes will move from advisory code comments to autonomously generating, testing, and committing patches for security flaws, failed builds, or runtime regressions without manual intervention (Ruotsalainen, 2025).
- 2. Cost-Aware Cognitive Compilation (Compiler.next).** Search-based, cost-aware compilation and build tools will dynamically optimize prompt tokens, foundation model parameters, and build hardware to balance accuracy, latency, and execution cost (Cogo et al., 2025).
- 3. Zero-Trust AI DevSecOps Governance.** Explainable AI (XAI) and deterministic guardrails within CI/CD engines will ensure that automated code changes comply with data privacy standards (e.g., ISO/IEC, CMMI), prevent adversarial data poisoning, and maintain an auditable lineage of AI-assisted code modifications.

9 Conclusion

This report showed how artificial intelligence extends DevOps into AIOps. Traditional DevOps automation is rule-based and struggles with data overload and reactive troubleshooting, while AIOps adds analysis, prediction, and decision support (Sections 2 to 4). In the delivery pipeline, AI supports test selection and failure analysis (Section 5), code review (Section 6), and cost optimization (Section 7), and Section 8 covered the implementation architecture, reported case-study results, and future directions. In every area the same limits apply: models depend on good data, a wrong prediction can let a real defect through, and humans must stay in control of high-impact decisions. AIOps is therefore best seen as a controlled extension of DevOps, not a replacement for engineers.

References

Amazon Web Services. (n.d.-a). *What is AIOps?* Retrieved September 20, 2026, from <https://aws.amazon.com/what-is/aiops/>

Amazon Web Services. (n.d.-b). *What is DevOps?* Retrieved September 20, 2026, from <https://aws.amazon.com/devops/what-is-devops/>

Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM*, 53(4), 50–58. <https://doi.org/10.1145/1721654.1721672>

Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., Hall, P., & Roberts, K. (2024). *Artificial intelligence risk management framework: Generative artificial intelligence profile* (NIST AI 600-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.600-1>

Bogatinovski, J., Nedelkoski, S., Acker, A., Schmidt, F., Wittkopp, T., Becker, S., Cardoso, J., & Kao, O. (2021). *Artificial intelligence for IT operations (AIOps) workshop white paper*. arXiv. <https://doi.org/10.48550/arXiv.2101.06054>

Cogo, F. R., Oliva, G. A., & Hassan, A. E. (2025). *Compiler.next: A search-based compiler to power the AI-native future of software engineering*. arXiv. <https://doi.org/10.48550/arxiv.2510.24799>

- Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)* (pp. 4–5). IEEE.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps: Building and scaling high performing technology organizations*. IT Revolution Press.
- GitHub. (2026a). *About GitHub Copilot code review*. GitHub Docs. <https://docs.github.com/en/copilot/concepts/agents/code-review>
- GitHub. (2026b). *Use GitHub Copilot code review across the pull request lifecycle*. GitHub Docs. <https://docs.github.com/en/copilot/tutorials/use-copilot-code-review-across-the-pull-request-lifecycle>
- GitHub. (2026c). *Using GitHub Copilot code review*. GitHub Docs. <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>
- Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.
- IBM. (n.d.). *What is AIOps?* Retrieved September 20, 2026, from <https://www.ibm.com/topics/aiops>
- Kakarla, R. (2024). AI-driven DevOps automation for CI/CD pipeline optimization. *Eastasouth Journal of Information System and Computer Science*, 2(1), 70–78.
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.
- Machalica, M., Samylkin, A., Porth, M., & Chandra, S. (2019). Predictive test selection. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (pp. 91–100). IEEE. <https://doi.org/10.1109/ICSE-SEIP.2019.00018>
- Mamun, M. N. H. (2024). Systemic integration of artificial intelligence and DevOps for scalable product development: A systematic literature review. *International Summit on Advanced Scientific Research*, 4(1), 1–32.
- Notaro, P., Cardoso, J., & Gerndt, M. (2021). A survey of AIOps methods for failure management. *ACM Transactions on Intelligent Systems and Technology*, 12(6), Article 81. <https://doi.org/10.1145/3483424>
- Ruotsalainen, E. (2025). *Systematic literature review of agentic AI and AIOps across the software development lifecycle*. Aalto University.
- Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 5, 3909–3943. <https://doi.org/10.1109/ACCESS.2017.2685629>
- Tabassi, E. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.100-1>